EXPERIMENT 14

Pro-Code Extension of a Zoho Creator Application

Using Deluge custom functions and Zoho Catalyst for an illustrative appointment priority-scoring workflow

Submitted by	Sri Janani B
Register number	241401108
Application	Patient Appointment Scheduling System
Platform	Zoho Creator (low-code) with Zoho Catalyst (pro-code)

1. Aim

To study how the Patient Appointment Scheduling System built in Zoho Creator can be extended beyond what standard workflow actions allow. The extension combines a reusable **Deluge custom function** inside Creator with a serverless **Zoho Catalyst** function that holds the more demanding logic. An appointment priority-scoring requirement is used as the worked example, showing how Creator sends data to an external service and uses the response. The code in this report is illustrative; it is not a record of a deployed implementation.

2. Procedure

Step	Activity
1. Identify the need	Review the existing patient form and workflows, and pick a requirement that built-in actions handle poorly: assigning a priority level to each appointment from the patient's age and existing conditions.
2. Compare options	Compare workflow actions, Deluge custom functions, Creator Custom APIs and Catalyst functions on skill needed and typical use (Section 3.1).
3. Design the architecture	Map the Creator form, workflow and custom function to a Catalyst Advanced I/O function through a managed connection (Section 3.2).
4. Write the Creator function	Package age and conditions as JSON and call the Catalyst endpoint with invokeurl, with a safe default if the call fails (Section 3.3).
5. Write the Catalyst function	Receive the payload, validate it, apply the example rules and return a JSON priority value (Section 3.4).
6. Plan the execution flow	Draw the sequence from form submission to record update, including the failure path (Section 3.5).
7. Decide and note precautions	Prepare a decision guide for choosing an extension type and list security, error-handling and cost precautions (Sections 3.6, 3.7).

Note: the endpoint URL and connection name are placeholders, and the code was neither deployed nor executed.

241401108

3. Output

3.1 Extension options compared

Option	Description	Skill level	Typical use
Workflow actions	Built-in actions such as notifications and record updates.	Low	Routine automation.
Deluge custom function	Reusable function called from Creator workflows.	Low to medium	Shared business rules inside Creator.
Creator Custom API	REST endpoint exposed by a Creator application.	Medium	External systems calling Creator.
Catalyst function	Serverless Node.js, Java or Python function invoked over HTTPS.	Medium to high	Complex logic, libraries, external systems.

3.2 Architecture of the extension

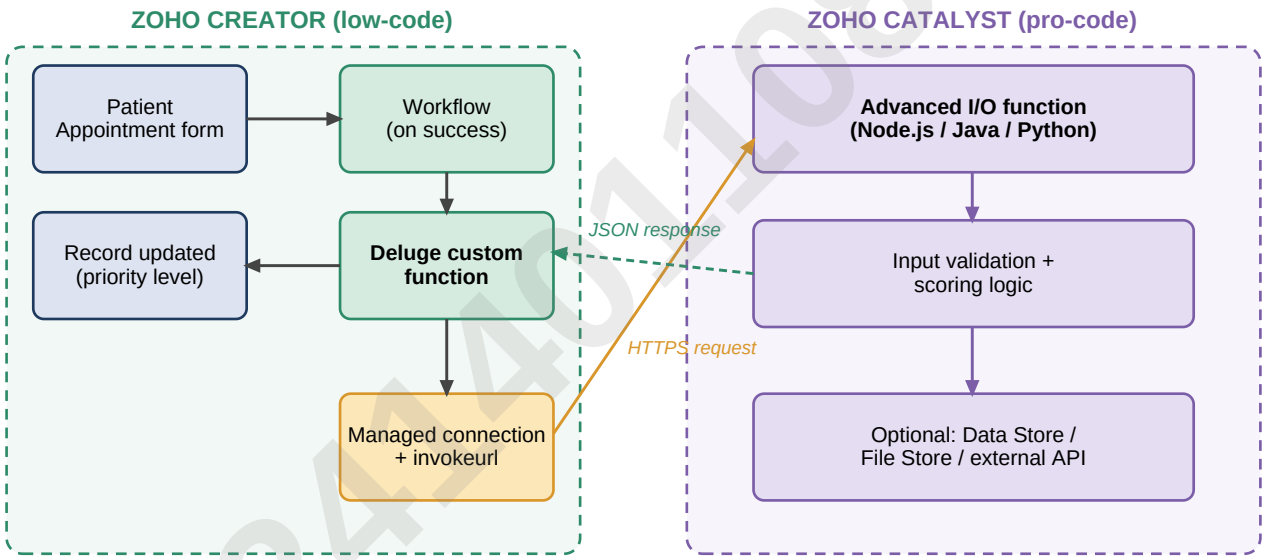

Fig. 1: Creator keeps the form and workflow; Catalyst handles the logic and returns JSON.

The Creator workflow calls a Deluge custom function after a successful submission. The function sends an HTTPS request through a managed connection to an Advanced I/O function on Catalyst. Catalyst validates the input, applies the scoring rules (optionally using its own storage or an external API) and returns a JSON response, which Creator uses to update the record.

3.3 Deluge custom function (illustrative)

```
// Standalone function: getPriorityScore
string getPriorityScore(int age, string conditions)
{
    priority = "Normal";            // default if anything fails
    try
    {
        payload = Map();
        payload.put("age", age);
        payload.put("conditions", conditions);
        response = invokeurl
        [
            url :"<Catalyst Advanced I/O endpoint>/score"
            type :POST
            parameters :payload.toString()
            headers :{"Content-Type":"application/json"}
            connection :"catalyst_connection"
        ];
        if(response.get("priority") != null)
        {
            priority = response.get("priority");
        }
    }
    catch (e)
    {
        info "Priority call failed: " + e;
    }
    return priority;
}

// Called from the workflow (on success of the form)
priority = thisapp.getPriorityScore(input.age, input.existing_conditions.toString());
Patient_Profile[ID == input.ID].priority_level = priority;
```

The function packages the request once so that any workflow can reuse it, and it falls back to the default priority when the call fails. Endpoint and connection names are placeholders; the request format, authentication, response shape and failure behaviour must be confirmed and tested before deployment.

3.4 Catalyst Advanced I/O function in Node.js (illustrative)

```
const express = require("express");
const app = express();
app.use(express.json());

app.post("/score", (req, res) => {
  const { age, conditions } = req.body || {};

  // validate input
  if (typeof age !== "number" || age < 0 || age > 120) {
    return res.status(400).json({ error: "Invalid age" });
  }
  const text = String(conditions || "").toLowerCase();

  // example scoring rules (programming demo only)
  let score = 0;
  if (age >= 60) score += 2;
  if (text.includes("heart disease")) score += 3;
  if (text.includes("diabetes")) score += 1;

  const priority = score >= 4 ? "High" : score >= 2 ? "Medium" : "Normal";
  res.status(200).json({ priority });
});

module.exports = app;
```

The function validates the request, adds example points for age 60 or above, a heart-disease entry and a diabetes entry, then maps the total to High, Medium or Normal. These thresholds are programming examples only and are **not clinical guidance**; real prioritisation rules must be approved by qualified medical staff.

241401108

3.5 Execution flow

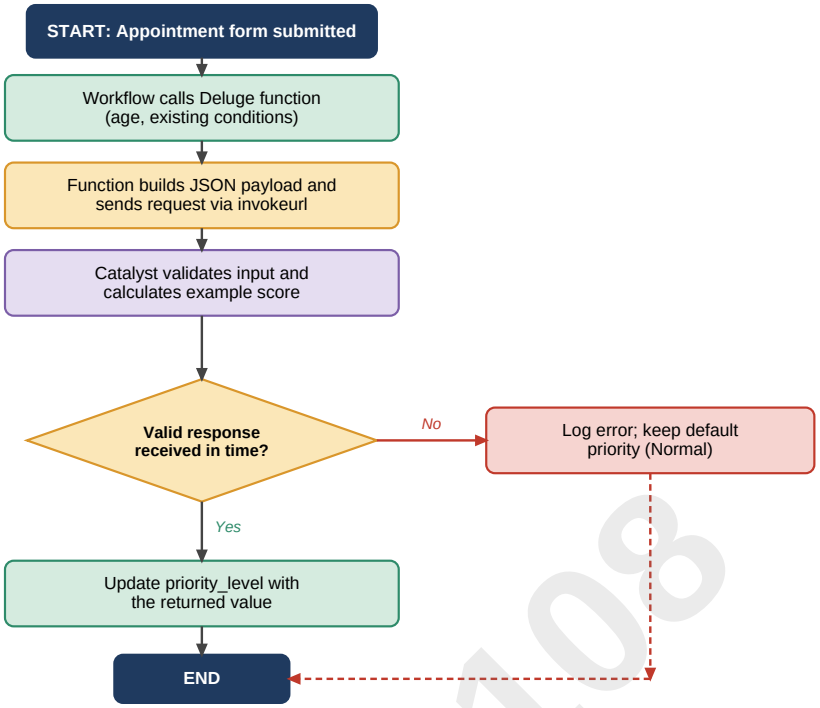

Fig. 2: Execution flow from form submission to record update, with the error path.

A failed external request must not cause the appointment to be lost. The design therefore keeps the default priority and records an error when no valid response arrives. A real implementation should also add timeouts and response validation.

3.6 Choosing an extension type

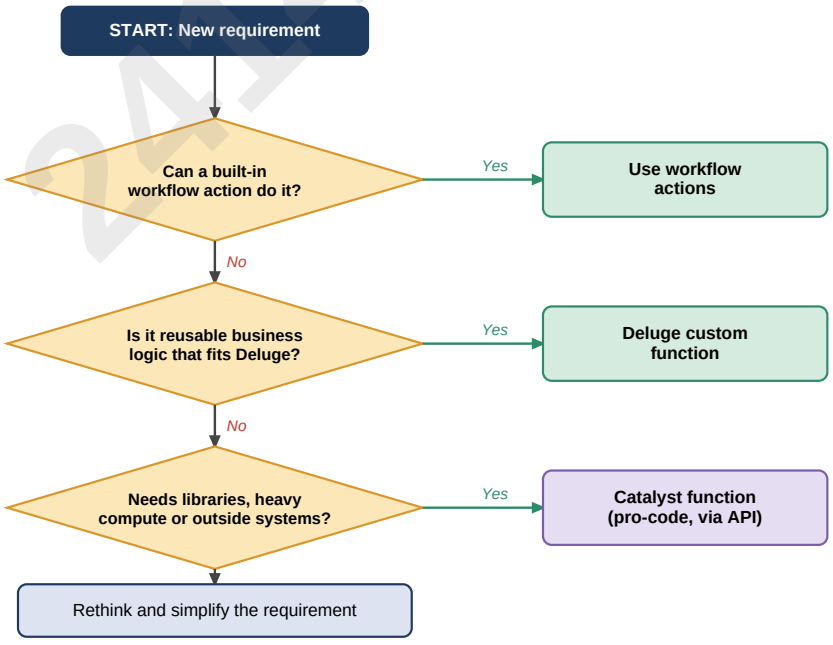

Fig. 3: Decision guide - prefer the simplest option that meets the requirement.

Question	Direction
Can a built-in action solve it?	Prefer a standard workflow action for simplicity.
Is the rule reusable within Creator?	Use a Deluge custom function.
Must an outside system call Creator?	Consider a Creator Custom API.

241401108

Question	Direction
Are libraries, heavy computation or external systems needed?	Consider a Catalyst function and review security and operational needs.
No option fits well?	Reconsider and simplify the requirement.

3.7 Benefits and precautions

Aspect	Benefit	Precaution
Flexibility	General-purpose languages and libraries support specialised logic.	More code means more testing and maintenance.
Reuse	One function can serve multiple workflows.	Keep inputs and outputs consistent.
Separation	External processing keeps Creator workflows lighter.	Handle network errors and timeouts.
Security	Managed connections keep credentials out of scripts.	Restrict access and validate inputs.
Limits and cost	Serverless infrastructure can scale execution.	Check current service limits and pricing.

4. Final Result

The study presents a two-part extension pattern: a reusable Deluge function inside Creator and a Catalyst serverless function reached through a managed HTTPS connection. It compares the available extension choices, outlines an illustrative scoring exchange with a safe failure path, and lists the security and error-handling precautions. Workflow actions and custom functions suffice for most needs, and Catalyst should be added only for complex logic, libraries or outside systems.

The exercise met its objective of exploring pro-code extension. The code was not deployed or tested, so no successful live run is claimed.

241401108